CS3001 Assignment-II: Reliable Transfer for a Disaster-Response Network

Course: CS3001 Computer Networks **Semester:** Fall 2026 ·

Type: Individual **Marks:** 100 **Due:** _____

After a major earthquake, a disaster-response organization links its Central Emergency Operations Center (EOC) to field rescue teams over a satellite link (**R = 2 Mbps, RTT = 500 ms, packet size L = 1,000 bytes = 8,000 bits**). The link loses, corrupts and delays packets and ACKs, and bandwidth is limited. Teams send casualty lists and supply requests; the EOC sends rescue assignments. A lost or corrupted packet could send a team to the wrong location, so every packet must arrive correctly and in order. You must design and test the transport protocol for this network.

Instructions

- Use Python, Java or C++. Do not use any ready-made GBN, SR or reliable-transport library.
- Use slide terminology (Chapter # 03): base, nextseqnum, expectedseqnum, rcv_base, cumulative ACK, sender/receiver window.
- Use the last four digits of your roll number as the random seed.
- Show all working and state your assumptions.

Task 1: Scenario & Protocol Decision (10 marks)

Answer each in 2–4 sentences, using the scenario.

1. Why is reliable data transfer important in this network?
2. Why can Stop-and-Wait be inefficient on this link?
3. Why is pipelining useful here?
4. Is GBN or SR more suitable for this scenario? Give two technical reasons.

Task 2: GBN vs SR Packet Trace (20 marks)

A rescue team transmits 10 numbered data packets (0 – 9) to the EOC. Window size N = 4; sequence numbers 0–9. Packet 2 is lost; packets 3 and 4 reach the receiver. All other packets and ACKs arrive correctly and in order, and ACKs follow the respective protocol rules. The timeout for packet 2 occurs before packet 2 is successfully received again.

Construct a step-by-step trace for both protocols, continuing until all 10 packets are delivered, using this table:

Event	Sender	Packet	Receiver	ACK	Window
1	Send pkt 0	0	…	…	[0–3]

(a) Go-Back-N: show packet transmissions, ACKs, receiver behaviour for out-of-order packets, the timeout, retransmitted packets and sender window movement.

(b) Selective Repeat: show packet transmissions, individual ACKs, receiver buffering, the timeout, retransmitted packet(s) and sender/receiver window movement.

Finally, answer: How many packets are retransmitted in GBN and in SR in this scenario, and why are they different?

Task 3: GBN & SR Implementation (40 marks)

Implement a GBN sender/receiver and an SR sender/receiver that communicate through a simulated unreliable channel you write yourself. rdt 3.0 is the foundation; no separate rdt 3.0 program is needed.

Configurable: protocol (gbn/sr), number of packets, window size N, sequence-number bits k (numbers wrap modulo 2^k), loss, corruption and delay probabilities, and timeout. Loss, corruption and delay apply to packets and ACKs; corruption must be detected with a checksum. Print a log line for every send, receive, ACK, loss, timeout and retransmission.

GBN must demonstrate	SR must demonstrate
Sliding window (base, nextseqnum)	Sliding sender and receiver windows
Cumulative ACKs	Individual ACKs
One timer for the oldest unACKed packet	A timer per packet (or equivalent)

On timeout, resend all outstanding packets	Receiver buffering of out-of-order packets
Receiver discards out-of-order packets	Resend only missing packets; deliver in order

Task 4: Experiments & Comparison (15 marks)

Run both protocols with 200 packets, N = 4 and k = 3 under:

A. Reliable channel (no loss or corruption);

B. Packet loss (loss 0.10);

C. Mixed errors (loss 0.05, corruption 0.05, delay 0.10).

For each run report packets transmitted, retransmissions, packets delivered successfully, and throughput or execution time. Give one results table and one graph comparing GBN and SR, with 3–4 sentences explaining the difference.

Task 5: Transport Layer Analysis (10 marks)

1. **Pipelining.** Explain why pipelining improves link utilization compared with Stop-and-Wait. Support your answer with U = (L/R) / (RTT + L/R) from the slides, for Stop-and-Wait and for N = 4 on the scenario link.
2. **Sequence numbers.** For SR with 3-bit sequence numbers: (a) how many sequence numbers are available; (b) what is the maximum safe window size; (c) why can a larger window cause ambiguity? Support (c) with a small diagram.

Deliverables

Submit one zip file RollNo_A2.zip (containing the following) along with small working video of your code (Share video as a drive link):

(1) source code;

(2) a short README with how to run, parameters and one sample command;

(3) the results table;

(4) one GBN vs SR graph;

(5) a report of at most 4 pages covering approach, implementation, experiments, results, comparison and conclusion, plus your answers to Tasks 1, 2 and 5.

Viva / Demonstration

A short viva/demo may be held. You may be asked to change the window size, increase packet loss, explain a retransmission, explain cumulative vs individual ACKs, explain receiver buffering, or explain why GBN retransmits more packets than SR. Marks for code you cannot explain may be reduced.

Marks

Task 1	Task 2	Task 3	Task 4	Task 5	Viva	Total
10	20	40	15	10	5	100

Student Rubric (100 marks)

Bands as a share of each criterion's marks: **Excellent** 90–100% · **Good** 70–89% · **Satisfactory** 40–69% · **Poor** below 40%. Criteria 2–4 = Task 3; criteria 5 + 7 = Task 4.

Criterion (marks)	Excellent	Good	Satisfactory	Poor
1. Conceptual Understanding (30) Tasks 1, 2	Scenario-based answers; both traces correct	One slip in a trace	Generic answers; one trace wrong	Traces missing; GBN/SR confused
2. GBN Implementation (15)	Cumulative ACK; one timer; go-back resend	Works; one minor bug	Works only without errors	Does not run or uses a library
3. SR Implementation (15)	Individual ACKs; buffering; selective resend	Works; one minor bug	Buffering or window wrong at times	Missing or behaves like GBN
4. Error Simulation (10)	Loss, corruption (checksum) and delay	One simplification	One or two error types only	No channel model
5. Experiments/Results (12)	A, B, C for both; table and labelled graph	Complete; graph weak	A run or graph missing	No results
6. Analysis (10) Task 5	Correct utilization values; N ≤ 4 with diagram	One small error	Partly correct; no diagram	Missing or wrong
7. Documentation (3)	Clear README and report ≤ 4 pages	Minor gaps	README or report incomplete	Missing
8. Viva (5)	Explains code and live changes	Needs a hint	Runs code, cannot explain	Cannot explain own code